LUIS ALEXIS CALVILLO DEVEZE
5/11/26

Sistema de control de clima en invernadero

Implementación de máquina de estados finitos (FSM) en PIC18F45K22

MPLAB XC8 · ADC · PWM · UART · I²C · LCD 2x16 · Teclado matricial

1. Descripción general del sistema

El sistema regula automáticamente la temperatura y ventilación de un invernadero utilizando el microcontrolador PIC18F45K22 como núcleo de control. La lógica de funcionamiento está organizada como una máquina de estados finitos (FSM) de cinco estados que se ejecuta de forma continua en el ciclo principal del programa.

El sensor LM35 proporciona una lectura analógica de temperatura que el ADC convierte a grados Celsius. Con ese valor, el sistema activa proporcionalmente el ventilador o el calefactor mediante señales PWM en los módulos CCP1 y CCP2. El operador puede ajustar los límites de temperatura desde el teclado matricial, observar el estado en el LCD y recibir un reporte en tiempo real por UART.

Opcionalmente, un sensor SHT21 conectado por I²C mide la humedad relativa, cuyo valor se muestra también en pantalla y se incluye en el reporte UART.

2. Hardware utilizado

Pin(es)	Periférico	Función	Configuración
RA0 / AN0	LM35	Lectura analógica de temperatura	ANSELA=0x01, ADCON2=0xAA
RC2 / CCP1	Motor ventilador	PWM proporcional a exceso de temperatura	CCP1CON=0x0C, PR2=249
RC1 / CCP2	Motor calefactor	PWM proporcional a deficit de temperatura	CCP2CON=0x0C, PR2=249
RD2–RD7	LCD 2x16	Visualizacion de estado en modo 4 bits	TRISD=0x00, RS=RD2, EN=RD3
RC6 / TX	UART	Reporte continuo a terminal (9600 bps)	TXSTA1=0x26, BAUDCON1=0x08
RC7 / RX	UART	Recepcion de datos serie	RCSTA1=0x90
RC3 / SCL	SHT21 (I²C)	Reloj I²C para sensor de humedad	SSPCON1=0x28, SSPADD=39
RC4 / SDA	SHT21 (I²C)	Datos I²C para sensor de humedad	TRISC4=1

RB4–RB7	Teclado 4x4	Filas del teclado matricial (salidas)	LATB=0xF0
RB0–RB3	Teclado 4x4	Columnas del teclado (entradas)	TRISB=0x0F, WPUB=0x0F
RE0	LED rojo	Alarma de temperatura alta	TRISE=0x00
RE1	LED azul	Alarma de temperatura baja	TRISE=0x00

3. Máquina de estados finitos (FSM)

La FSM se implementa en el ciclo principal mediante un switch-case sobre la variable global estado_actual. Cada estado realiza una tarea bien delimitada y transfiere el control al siguiente estado al finalizar.

ESTADO_INIT — Inicialización

Es el punto de entrada al sistema. Se configuran todos los periféricos en el siguiente orden: oscilador, GPIO, ADC, PWM (Timer2 + CCP1/CCP2), UART, I²C y LCD. El oscilador interno se ajusta a 16 MHz mediante OSCCON=0x70. Una vez completada la inicialización, el LCD muestra el mensaje de bienvenida y el sistema avanza a ESTADO_LECTURA. Este estado se ejecuta una sola vez en toda la operación.

ESTADO_LECTURA — Adquisición de datos

El ADC convierte la señal del LM35 en el canal AN0. La tensión de salida del LM35 es 10 mV por grado Celsius con referencia VSS, por lo que la conversión a temperatura se realiza como:

```
temperatura = (raw * 500.0) / 1023.0   // Vref=5V, 10mV/degC
```

Simultáneamente se consulta el SHT21 por I²C para obtener la humedad relativa, y se procesa el teclado matricial para detectar si el operador desea cambiar algún límite de temperatura. Al terminar, el estado avanza a ESTADO_COMPARACION.

ESTADO_COMPARACION — Lógica de control

Se compara la temperatura medida contra los umbrales temp_max y temp_min configurables. El sistema aplica una respuesta proporcional: si la temperatura supera el máximo, el duty cycle del ventilador aumenta en proporción al exceso (hasta 10 °C sobre el límite). Si está por debajo del mínimo, el calefactor recibe el duty proporcional al déficit.

```
// Temperatura alta -> ventilador proporcional
exceso = temperatura - temp_max;           // diferencia sobre el limite
if (exceso > 10.0) exceso = 10.0;          // clamp a 10 grados maximo
duty_ventilador = (exceso / 10.0) * 1000;  // 0-1000 = 0-100%

// Temperatura baja -> calefactor proporcional
deficit = temp_min - temperatura;
duty_calefactor = (deficit / 10.0) * 1000;
```

Cuando la temperatura se encuentra dentro del rango, ambos duty cycles quedan en cero y los LEDs de alarma se apagan. Si hay alarma alta se enciende RE0; si hay alarma baja, RE1.

ESTADO_ACCION — Aplicación de PWM

Los valores calculados en el estado anterior se transfieren a los registros CCPR1L/CCP1CON y CCPR2L/CCP2CON. La frecuencia del PWM es de 16 kHz (PR2=249, prescaler 1:1, Fosc=16 MHz), lo cual permite una variación suave de velocidad sin ruido audible en los motores DC.

```
// Escala duty 0-1000 a los 10 bits del CCP
val10 = (duty * (PR2 + 1)) / 1000;   // 0-250 para PR2=249
CCPR1L = val10 >> 2;                  // bits 9:2
CCP1CON.DC1B1 = (val10 >> 1) & 1;    // bit 1
CCP1CON.DC1B0 = val10 & 1;            // bit 0
```

ESTADO_REPORTE — Visualización y comunicación

Se actualiza el LCD con dos líneas: la superior muestra temperatura y humedad; la inferior indica el porcentaje activo del ventilador o calefactor, o el mensaje RANGO NORMAL si no hay activación. Tras actualizar el LCD, se envía por UART una cadena con todos los parámetros del sistema, útil para monitoreo en la terminal virtual de Proteus.

```
// Linea 0 del LCD
sprintf(linea, "T:%.1fC H:%.1f%%  ", temperatura, humedad);

// Reporte UART
sprintf(buf, "TEMP:%.2f HUME:%.2f VEN:%u CAL:%u TMAX:%.1f TMIN:%.1f\r\n",
        temperatura, humedad, duty_ven, duty_cal, temp_max, temp_min);
```

Después de una pausa de 500 ms, el estado regresa a ESTADO_LECTURA, completando el ciclo.

4. Configuración de periféricos clave

Oscilador interno a 16 MHz

El PIC18F45K22 se configura para usar su oscilador RC interno a máxima velocidad. OSCCON=0x70 selecciona IRCF=111 (16 MHz) con SCS=00 (reloj primario). El programa espera el bit HFIOFS antes de continuar para garantizar estabilidad.

ADC — conversión del LM35

Se utiliza el módulo ADC del PIC con justificación a la derecha (ADFM=1), tiempo de adquisición de 4 TAD y reloj Fosc/16, resultando en TAD=1 µs que cumple el mínimo del PIC. La referencia de voltaje es VDD (5 V) y VSS, lo que permite la fórmula de conversión directa mostrada en la sección 3.

PWM mediante Timer2 y módulos CCP

Parámetro	Valor	Resultado
Frecuencia del oscilador	`Fosc = 16 MHz`	—
Registro PR2	`249`	—
Prescaler Timer2	`1:1`	—
Frecuencia PWM	`Fosc / (4 × (PR2+1) × pre)`	16 kHz
Resolución	`10 bits (0–1000 en el código)`	0.1% por paso

UART a 9600 bps

Con BRG16=1 y BRGH=1 el generador de baudrate usa la fórmula Baud = Fosc / (4 × (BRG+1)). Con BRG=0x01A0 (416 decimal) se obtienen 9592 bps, un error de 0.08 % respecto a 9600 bps, dentro del margen aceptable para comunicación serie.

I²C y sensor SHT21

El módulo MSSP se configura en modo Master I²C con SSPADD=39, resultando en SCL = 16 MHz / (4 × 40) = 100 kHz. El protocolo de lectura del SHT21 envía el comando de medición sin hold (0xF5), espera 30 ms para la conversión de 12 bits y lee 2 bytes. La fórmula de conversión de la hoja de datos es:

```
humedad = -6.0 + 125.0 * (raw / 65536.0)   // resultado en %RH
```

La función i2c_write incluye un contador de timeout de 5000 ciclos para evitar que el sistema se bloquee si el sensor no responde. Si se detecta NACK o timeout, la función retorna -1.0 % como indicador de fallo.

Teclado matricial 4x4

El escaneo se realiza activando una fila a la vez (poniendo en bajo RB4, RB5, RB6 o RB7 sucesivamente) y leyendo las cuatro columnas (RB0–RB3). Los pull-ups internos del PORTB están activos en las columnas para garantizar nivel alto cuando no se presiona ninguna tecla. La función teclado_procesar interpreta las teclas A y B como inicio de configuración de TMAX y TMIN respectivamente, los dígitos 0–9 acumulan el valor, y # confirma el nuevo límite.

5. Operación del sistema

Configuración de umbrales desde el teclado

Tecla física	Código interno	Acción
`÷ (fila 1, col 4)`	`'A'`	Inicia captura de TMAX. LCD muestra: TMax:
`× (fila 2, col 4)`	`'B'`	Inicia captura de TMIN. LCD muestra: TMin:
`7 8 9 / 4 5 6 / 1 2 3 / 0`	`'1'–'9','0'`	Dígitos que acumulan el valor numérico
`= (fila 4, col 3)`	`'#'`	Confirma y guarda el valor ingresado

Ejemplo: para fijar TMAX = 28 °C se presiona ÷, luego 2, luego el 8, y finalmente = para confirmar.

Comportamiento de salidas

Condición	Ventilador	Calefactor	LEDs
T dentro del rango	Apagado (0%)	Apagado (0%)	Ambos apagados
T > TMAX (exceso ≤10°C)	Proporcional 0–100%	Apagado	RE0 encendido
T > TMAX (exceso >10°C)	100% constante	Apagado	RE0 encendido
T < TMIN (deficit ≤10°C)	Apagado	Proporcional 0–100%	RE1 encendido

6. Código fuente completo

A continuación se presenta el programa completo en C para MPLAB XC8.

```
/*
 * Hardware:
 * LM35    -> AN0 (RA0)
 * CCP1    -> RC2 (Ventilador PWM)
 * CCP2    -> RC1 (Calefactor PWM)
 * LCD     -> Puerto D + E(RD3) RS(RD2) RW=GND (modo 4 bits, datos RD4-RD7)
 * UART    -> TX=RC6, RX=RC7
 * I2C     -> SDA=RC4, SCL=RC3
 * Teclado-> Filas RB4-RB7, Columnas RB0-RB3
 * LED Alarma Alta  -> RE0
 * LED Alarma Baja  -> RE1
 */

// ============================================================
//  Configuracion de fuses
// ============================================================
#pragma config FOSC   = INTIO67
#pragma config PLLCFG = OFF
#pragma config PRICLKEN = ON
#pragma config FCMEN  = OFF
#pragma config IESO   = OFF
#pragma config PWRTEN = ON
#pragma config BOREN  = OFF
#pragma config WDTEN  = OFF
#pragma config PBADEN = OFF
#pragma config MCLRE  = EXTMCLR
#pragma config STVREN = ON
#pragma config LVP    = OFF
#pragma config DEBUG  = OFF
#pragma config CP0    = OFF
#pragma config CP1    = OFF
#pragma config CPB    = OFF
#pragma config WRT0   = OFF
#pragma config WRT1   = OFF
#pragma config WRTB   = OFF
#pragma config WRTC   = OFF
#pragma config EBTR0  = OFF
#pragma config EBTR1  = OFF
#pragma config EBTRB  = OFF
```

```
// ============================================================
//  Includes y frecuencia
// ============================================================
#include <xc.h>
#include <stdint.h>
#include <stdio.h>
#include <string.h>

#define _XTAL_FREQ  16000000UL   // 16 MHz interno

// ============================================================
//  LEDs de alarma
// ============================================================
#define LED_ALTA    LATEbits.LATE0   // RE0 -> temp alta
#define LED_BAJA    LATEbits.LATE1   // RE1 -> temp baja

// ============================================================
//  Direccion I2C del sensor de humedad (SHT21 = 0x40)
// ============================================================
#define SHT21_ADDR  0x40

// ============================================================
//  Parametros por defecto
// ============================================================
#define TEMP_MAX_DEFAULT  30.0f
#define TEMP_MIN_DEFAULT  15.0f

// ============================================================
//  Estados de la FSM
// ============================================================
typedef enum {
    ESTADO_INIT = 0,
    ESTADO_LECTURA,
    ESTADO_COMPARACION,
    ESTADO_ACCION,
    ESTADO_REPORTE
} Estado_t;

// ============================================================
//  Variables globales
// ============================================================
volatile Estado_t estado_actual = ESTADO_INIT;

float temperatura    = 0.0f;
float humedad        = 0.0f;
float temp_max       = TEMP_MAX_DEFAULT;
float temp_min       = TEMP_MIN_DEFAULT;
float   valor_acumulado = 0.0f;
uint8_t modo_config     = 0;   // 0=normal, 'A'=max, 'B'=min

uint16_t duty_ventilador  = 0;
uint16_t duty_calefactor  = 0;

char    uart_buf[64];
uint8_t teclado_tecla = 0xFF;

// ============================================================
//  PROTOTIPOS
// ============================================================
void init_oscilador(void);
void init_gpio(void);
void init_adc(void);
void init_pwm(void);
```

```
void init_uart(void);
void init_i2c(void);
void init_lcd(void);
void lcd_nibble(uint8_t nib, uint8_t rs);
void lcd_cmd(uint8_t cmd);
void lcd_char(char c);
void lcd_str(const char *s);
void lcd_goto(uint8_t row, uint8_t col);
void lcd_clear(void);
uint16_t adc_leer(uint8_t canal);
float    adc_a_celsius(uint16_t raw);
void pwm_set_ventilador(uint16_t duty);
void pwm_set_calefactor(uint16_t duty);
void uart_send_char(char c);
void uart_send_str(const char *s);
void    i2c_start(void);
void    i2c_stop(void);
uint8_t i2c_write(uint8_t dato);
uint8_t i2c_read(uint8_t ack);
float   sht21_leer_humedad(void);
uint8_t teclado_escanear(void);
void    teclado_procesar(void);
void fsm_init(void);
void fsm_lectura(void);
void fsm_comparacion(void);
void fsm_accion(void);
void fsm_reporte(void);
void delay_ms(uint16_t ms);

// ============================================================
//  MAIN
// ============================================================
int main(void) {
    estado_actual = ESTADO_INIT;

    while (1) {
        switch (estado_actual) {
            case ESTADO_INIT:        fsm_init();        break;
            case ESTADO_LECTURA:     fsm_lectura();     break;
            case ESTADO_COMPARACION: fsm_comparacion(); break;
            case ESTADO_ACCION:      fsm_accion();      break;
            case ESTADO_REPORTE:     fsm_reporte();     break;
            default:
                estado_actual = ESTADO_INIT;
                break;
        }
    }
    return 0;
}

// ============================================================
//  FSM - ESTADOS
// ============================================================
void fsm_init(void) {
    init_oscilador();
    init_gpio();
    init_adc();
    init_pwm();
    init_uart();
    init_i2c();
    init_lcd();

    lcd_clear();
```

```
    lcd_goto(0, 0); lcd_str("Invernadero FSM");
    lcd_goto(1, 0); lcd_str("Iniciando...");
    delay_ms(1500);
    lcd_clear();

    uart_send_str("Sistema iniciado\r\n");

    temp_max = TEMP_MAX_DEFAULT;
    temp_min = TEMP_MIN_DEFAULT;

    estado_actual = ESTADO_LECTURA;
}

void fsm_lectura(void) {
    uint16_t raw;

    raw         = adc_leer(0);
    temperatura = adc_a_celsius(raw);
    humedad     = sht21_leer_humedad();
    teclado_procesar();

    estado_actual = ESTADO_COMPARACION;
}

void fsm_comparacion(void) {
    duty_ventilador = 0;
    duty_calefactor = 0;
    LED_ALTA = 0;
    LED_BAJA = 0;

    if (temperatura > temp_max) {
        float exceso = temperatura - temp_max;
        if (exceso > 10.0f) exceso = 10.0f;
        duty_ventilador = (uint16_t)(exceso / 10.0f * 1000.0f);
        LED_ALTA = 1;
    } else if (temperatura < temp_min) {
        float deficit = temp_min - temperatura;
        if (deficit > 10.0f) deficit = 10.0f;
        duty_calefactor = (uint16_t)(deficit / 10.0f * 1000.0f);
        LED_BAJA = 1;
    }
    estado_actual = ESTADO_ACCION;
}

void fsm_accion(void) {
    pwm_set_ventilador(duty_ventilador);
    pwm_set_calefactor(duty_calefactor);
    estado_actual = ESTADO_REPORTE;
}

void fsm_reporte(void) {
    char linea[17];

    // SOLO actualizar la LCD si NO estamos en modo configuración
    if (modo_config == 0) {
        lcd_goto(0, 0);
        sprintf(linea, "T:%.1fC H:%.1f%%  ", temperatura, humedad);
        lcd_str(linea);

        lcd_goto(1, 0);
        if (duty_ventilador > 0) {
            sprintf(linea, "VEN:%3u%%  CAL:---", (unsigned)(duty_ventilador / 10));
        } else if (duty_calefactor > 0) {
```

```
            sprintf(linea, "VEN:---  CAL:%3u%%", (unsigned)(duty_calefactor / 10));
        } else {
            lcd_str("   RANGO NORMAL ");
            goto skip_lcd_line2;
        }
        lcd_str(linea);
        skip_lcd_line2:;
    }

    // UART siempre se actualiza (para monitoreo interno)
    sprintf(uart_buf,
            "TEMP:%.2f HUME:%.2f VEN:%u CAL:%u TMAX:%.1f TMIN:%.1f\r\n",
            temperatura, humedad,
            (unsigned)duty_ventilador, (unsigned)duty_calefactor,
            temp_max, temp_min);
    uart_send_str(uart_buf);

    delay_ms(500);
    estado_actual = ESTADO_LECTURA;
}

// ============================================================
//  INICIALIZACION DE PERIFERICOS
// ============================================================
void init_oscilador(void) {
    OSCCON = 0x70;              // IRCF=111 -> 16 MHz, SCS=00 -> primary clock
    while (!OSCCONbits.HFIOFS);
}

void init_gpio(void) {
    ANSELA = 0x01;   // solo RA0 analogico (LM35)
    ANSELB = 0x00;
    ANSELC = 0x00;
    ANSELD = 0x00;
    ANSELE = 0x00;

    TRISD = 0x00;
    LATD  = 0x00;

    TRISE = 0x00;
    LATE  = 0x00;

    // configurar TRISB. Esto evita que las columnas del teclado (RB0-RB3)
    // floten en hardware real. En Proteus el keypad los simula, pero en
    // una PCB real sin pull-ups se obtienen lecturas falsas.
    INTCON2bits.RBPU = 0;   // 0 = pull-ups habilitados en PORTB
    WPUB = 0x0F;             // pull-up activo solo en RB0-RB3 (columnas)

    TRISB = 0x0F;    // RB0-RB3 entradas (columnas), RB4-RB7 salidas (filas)
    LATB  = 0xF0;    // todas las filas en alto (reposo)

}

void init_adc(void) {
    ADCON0 = 0x00;   // canal AN0, ADC apagado aun
    ADCON1 = 0x00;   // Vref+ = VDD, Vref- = VSS
    // TAD = 1/( 16 MHz / 16 ) = 1 us  ->  cumple el minimo de 0.7 us
    ADCON2 = 0xAA;   // ADFM=1 (der.), ACQT=010 (4 Tad), ADCS=101 (Fosc/16)
    ADCON0bits.ADON = 1;
}

void init_pwm(void) {
    // Timer2: prescaler 1:1, PR2=249
```

```
    // Frecuencia PWM = 16 MHz / (4 * 250 * 1) = 16 kHz
    T2CON = 0x02;
    PR2   = 249;
    TMR2  = 0;
    T2CONbits.TMR2ON = 1;

    // CCP1 -> RC2 (ventilador)
    TRISCbits.TRISC2 = 0;
    CCP1CON  = 0x0C;        // modo PWM
    CCPR1L   = 0;
    CCP1CONbits.DC1B0 = 0;
    CCP1CONbits.DC1B1 = 0;

    // CCP2 -> RC1 (calefactor)
    TRISCbits.TRISC1 = 0;
    CCP2CON  = 0x0C;        // modo PWM
    CCPR2L   = 0;
    CCP2CONbits.DC2B0 = 0;
    CCP2CONbits.DC2B1 = 0;
}

void init_uart(void) {
    TRISCbits.TRISC6 = 0;   // TX salida
    TRISCbits.TRISC7 = 1;   // RX entrada
    // BRG16=1, BRGH=1: Baud = 16 MHz / (4 * (BRG+1))
    // BRG = 0x01A0 = 416  ->  16 000 000 / (4 * 417) = 9592 bps ~ 9600 bps
    SPBRGH1  = 0x01;
    SPBRG1   = 0xA0;
    TXSTA1   = 0x26;    // BRGH=1, SYNC=0, TXEN=1
    RCSTA1   = 0x90;    // SPEN=1, CREN=1
    BAUDCON1 = 0x08;    // BRG16=1
}

void init_i2c(void) {
    // Apagamos el módulo I2C de hardware para que no nos estorbe
    SSPCON1 = 0x00;
    LATCbits.LATC3 = 0;
    LATCbits.LATC4 = 0;
    TRISCbits.TRISC3 = 1; // SCL suelto (1 lógico)
    TRISCbits.TRISC4 = 1; // SDA suelto (1 lógico)
}

// ============================================================
//  LCD (modo 4 bits, RD7..RD4=D7..D4, EN=RD3, RS=RD2, RW=GND)
// ============================================================
#define LCD_PORT    LATD
#define LCD_EN      LATDbits.LATD3
#define LCD_RS      LATDbits.LATD2

void lcd_nibble(uint8_t nib, uint8_t rs) {
    LCD_PORT = (LCD_PORT & 0x0B) | ((nib & 0x0F) << 4) | (rs ? 0x04 : 0x00);
    LCD_EN = 1;
    __delay_us(2);
    LCD_EN = 0;
    __delay_us(50);
}

void lcd_cmd(uint8_t cmd) {
    lcd_nibble(cmd >> 4, 0);
    lcd_nibble(cmd & 0x0F, 0);
    if (cmd < 4) __delay_ms(2);
}
```

```
void lcd_char(char c) {
    lcd_nibble((uint8_t)c >> 4, 1);
    lcd_nibble((uint8_t)c & 0x0F, 1);
}

void lcd_str(const char *s) {
    while (*s) lcd_char(*s++);
}

void lcd_goto(uint8_t row, uint8_t col) {
    uint8_t addr = (row == 0) ? 0x80 : 0xC0;
    lcd_cmd(addr + col);
}

void lcd_clear(void) {
    lcd_cmd(0x01);
    __delay_ms(2);
}

void init_lcd(void) {
    __delay_ms(50);
    lcd_nibble(0x03, 0); __delay_ms(5);
    lcd_nibble(0x03, 0); __delay_us(150);
    lcd_nibble(0x03, 0); __delay_us(150);
    lcd_nibble(0x02, 0);   // cambiar a 4 bits
    lcd_cmd(0x28);         // 2 lineas, fuente 5x8
    lcd_cmd(0x0C);         // display on, cursor off
    lcd_cmd(0x06);         // incremento automatico, sin desplazamiento
    lcd_clear();
}

// ============================================================
//  ADC
// ============================================================
uint16_t adc_leer(uint8_t canal) {
    ADCON0bits.CHS = canal;
    __delay_us(20);         // tiempo de adquisicion
    ADCON0bits.GO  = 1;
    while (ADCON0bits.GO);
    return ((uint16_t)ADRESH << 8) | ADRESL;
}

// LM35: 10 mV/grado, Vref=5V
// Temperatura = (raw * 5000 mV) / (1023 * 10 mV/grado) = raw*500/1023
float adc_a_celsius(uint16_t raw) {
    return ((float)raw * 500.0f) / 1023.0f;
}

// ============================================================
//  PWM (duty 0-1000 representa 0-100.0%)
// ============================================================
void pwm_set_ventilador(uint16_t duty) {
    if (duty > 1000) duty = 1000;
    uint16_t val10 = (uint16_t)(((uint32_t)duty * (PR2 + 1)) / 1000);
    CCPR1L = (uint8_t)(val10 >> 2);
    CCP1CONbits.DC1B1 = (val10 >> 1) & 0x01;
    CCP1CONbits.DC1B0 = val10 & 0x01;
}

void pwm_set_calefactor(uint16_t duty) {
    if (duty > 1000) duty = 1000;
    uint16_t val10 = (uint16_t)(((uint32_t)duty * (PR2 + 1)) / 1000);
    CCPR2L = (uint8_t)(val10 >> 2);
```

```
    CCP2CONbits.DC2B1 = (val10 >> 1) & 0x01;
    CCP2CONbits.DC2B0 = val10 & 0x01;
}

// ============================================================
//  UART
// ============================================================
void uart_send_char(char c) {
    while (!TXSTA1bits.TRMT);
    TXREG1 = (uint8_t)c;
}

void uart_send_str(const char *s) {
    while (*s) uart_send_char(*s++);
}

// ============================================================
//  I2C por Software (Bit-Bang) - El Hack anti-Proteus
// ============================================================
#define SCL_TRIS TRISCbits.TRISC3
#define SDA_TRIS TRISCbits.TRISC4
#define SCL_PIN  PORTCbits.RC3
#define SDA_PIN  PORTCbits.RC4

void i2c_start(void) {
    SDA_TRIS = 1; SCL_TRIS = 1; __delay_us(10);
    SDA_TRIS = 0; __delay_us(10); // Bajar Datos
    SCL_TRIS = 0; __delay_us(10); // Bajar Reloj
}

void i2c_stop(void) {
    SDA_TRIS = 0; SCL_TRIS = 0; __delay_us(10);
    SCL_TRIS = 1; __delay_us(10); // Subir Reloj
    SDA_TRIS = 1; __delay_us(10); // Subir Datos
}

uint8_t i2c_write(uint8_t dato) {
    uint8_t i, ack;
    for (i = 0; i < 8; i++) {
        if (dato & 0x80) SDA_TRIS = 1; else SDA_TRIS = 0;
        __delay_us(10);
        SCL_TRIS = 1; __delay_us(10); // Reloj Arriba
        SCL_TRIS = 0; __delay_us(10); // Reloj Abajo
        dato <<= 1;
    }
    SDA_TRIS = 1; // Soltamos para escuchar la respuesta
    __delay_us(10);
    SCL_TRIS = 1; __delay_us(10);
    ack = SDA_PIN; // Leemos si el SHT21 respondió
    SCL_TRIS = 0; __delay_us(10);
    return ack;
}

uint8_t i2c_read(uint8_t manda_ack) {
    uint8_t i, dato = 0;
    SDA_TRIS = 1; // Soltar la línea
    for (i = 0; i < 8; i++) {
        dato <<= 1;
        SCL_TRIS = 1; __delay_us(10);
        if (SDA_PIN) dato |= 1; // Atrapamos el bit
        SCL_TRIS = 0; __delay_us(10);
    }
    // Respondemos ACK (0) o NACK (1)
```

```
    if (manda_ack) SDA_TRIS = 0; else SDA_TRIS = 1;
    __delay_us(10);
    SCL_TRIS = 1; __delay_us(10);
    SCL_TRIS = 0; __delay_us(10);
    SDA_TRIS = 1;
    return dato;
}

// ============================================================
//  SHT21 - Lectura de humedad relativa
// ============================================================
float sht21_leer_humedad(void) {
    uint16_t raw;

    i2c_start();
    if (i2c_write((SHT21_ADDR << 1) | 0x00)) {
        i2c_stop();
        return -1.0f;
    }

    i2c_write(0xF5);
    i2c_stop();

    // Tiempo para que el sensor mida
    __delay_ms(80);

    i2c_start();
    if(i2c_write((SHT21_ADDR << 1) | 0x01)) {
         i2c_stop();
         return -2.0f; // Si sale esto, el sensor no quiso responder
    }

    raw  = ((uint16_t)i2c_read(1) << 8); // Lee MSB
    raw |= (i2c_read(0) & 0xFC);         // Lee LSB

    i2c_stop();

    return -6.0f + 125.0f * ((float)raw / 65536.0f);
}


// ============================================================
//  TECLADO MATRICIAL 4x4
// ============================================================
static const char mapa_teclas[4][4] = {
    {'7','8','9','A'},
    {'4','5','6','B'},
    {'1','2','3','C'},
    {'*','0','#','D'}
};

uint8_t teclado_escanear(void) {
    for (uint8_t fila = 0; fila < 4; fila++) {
        LATB = ~(0x10 << fila) & 0xF0;   // poner fila actual en bajo
        __delay_us(10);
        uint8_t cols = PORTB & 0x0F;
        if (cols != 0x0F) {              // alguna columna bajo -> tecla presionada
            for (uint8_t col = 0; col < 4; col++) {
                if (!(cols & (1 << col))) {
                    return (uint8_t)mapa_teclas[fila][col];
                }
            }
        }
```

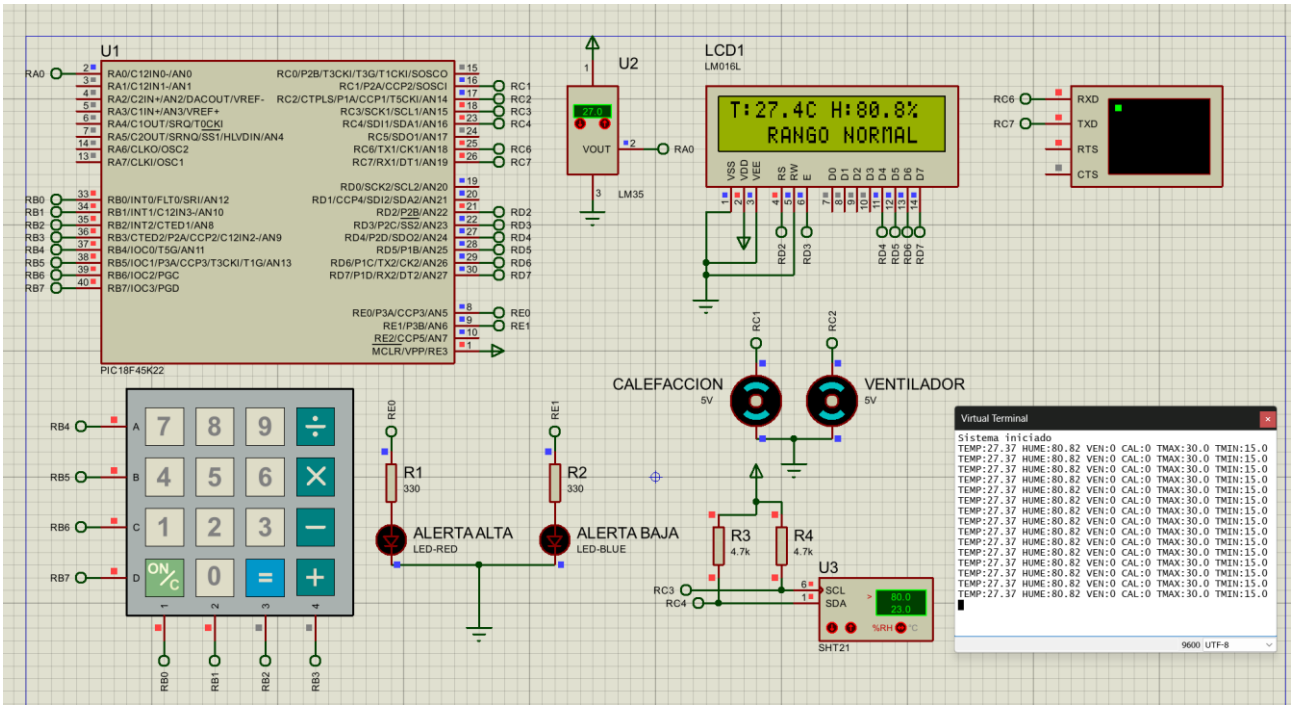
```
    }
    LATB = 0xF0;   // reposo: todas las filas en alto
    return 0xFF;   // sin tecla
}


void teclado_procesar(void) {
    uint8_t tecla = teclado_escanear();
    if (tecla == 0xFF) return;

    if (tecla == 'A') { // Botón División (÷)
        modo_config = 'A';
        valor_acumulado = 0.0f;
        lcd_goto(1, 0); lcd_str("Conf TMax:      ");
        return;
    }
    if (tecla == 'B') { // Botón Multiplicación (x)
        modo_config = 'B';
        valor_acumulado = 0.0f;
        lcd_goto(1, 0); lcd_str("Conf TMin:      ");
        return;
    }
    if (tecla == '#' && modo_config != 0) { // Botón Igual (=)
        if (modo_config == 'A') temp_max = valor_acumulado;
        else                    temp_min = valor_acumulado;
        modo_config = 0; // Salir de configuración
        valor_acumulado = 0.0f;
        lcd_clear(); // Limpiar para que la FSM pinte fresco
        return;
    }
    if (tecla >= '0' && tecla <= '9' && modo_config != 0) {
        valor_acumulado = valor_acumulado * 10.0f + (tecla - '0');
        // Mostrar el número que se va escribiendo
        char temp_str[10];
        sprintf(temp_str, "%.0f ", valor_acumulado);
        lcd_goto(1, 11);
        lcd_str(temp_str);
    }
    __delay_ms(150);   // Anti-rebote
}

// ============================================================
//  UTILIDADES
// ============================================================
void delay_ms(uint16_t ms) {
    while (ms--) __delay_ms(1);
}
```

7. Diagrama de circuito en Proteus



8. Video demostrativo

El video se encuentra en el siguiente link:
https://youtu.be/6XPwi0WVWms